

TEZPUR UNIVERSITY

Networking Internship Program

Assignment 7

Secure Network Application Development Using TCP

Submitted by

Bhargav Jyoti Saikia

Roll Number: CSB24022

Department of Computer Science and Engineering
Tezpur University

Submission Date: July 15, 2026

Table of Contents

1. Objective	3
2. Introduction	3
2.1 Authentication and Authorization	3
2.2 Why Passwords Must Never Be Stored in Plaintext	3
2.3 Threats Addressed in This Assignment	3
3. Software and System Requirements	3
4. Network Topology	4
5. Authentication Flow	4
5.1 Login Window	4
5.2 Successful Authentication	5
6. Security Features Implemented	6
6.1 User Authentication	6
6.2 Secure Password Storage	6
6.3 Duplicate Login Prevention	7
6.4 Input Validation	8
6.5 Failed Login Protection	9
6.6 Session Management and Secure Logging	10
7. Components Reused from Assignment 6	11
8. Implementation Design Decisions	11
9. Testing Results	11
9.1 Security Log Walkthrough	12
9.2 Functional Chat Verification	12
9.3 Handling Client-Side Feedback	13
10. Wireshark Verification	14
11. Handwritten Reflection	16
12. Conclusion	16
13. References	16

1. Objective

The objective of Assignment 7 is to enhance the GUI-based multi-client TCP chat application developed in Assignment 6 by implementing practical application-level security mechanisms. Rather than building a new application, this assignment extends the existing `client_gui.py` and `server.py` with authentication, secure password storage, session management, and input validation, keeping the emphasis on applied security engineering rather than advanced cryptography.

The specific goals of this assignment are to require a username and password before a client is admitted to the chat, store passwords as SHA-256 hashes rather than plaintext, prevent the same username from holding two simultaneous sessions, reject malformed usernames, oversized messages, and unsupported commands, temporarily block an account after repeated failed logins, enforce an inactivity session timeout with clean logout handling, maintain a password-free security audit log, and verify the resulting behaviour using Wireshark packet captures on port 5000.

2. Introduction

2.1 Authentication and Authorization

Authentication is the process of confirming that a user is who they claim to be, typically by verifying a shared secret such as a password. Authorization is a separate, subsequent step that determines what an already-authenticated user is permitted to do. In this assignment, the server authenticates a client by checking a submitted username/password pair against a stored credential set before a TCP session is allowed to exchange chat data; authorization in this system is simple and implicit, since every authenticated user is granted the same set of chat privileges (broadcast, private messaging, and the `/list` command).

2.2 Why Passwords Must Never Be Stored in Plaintext

If a credential store holds plaintext passwords, anyone who gains read access to that file, whether through a server compromise, a misconfigured backup, or an insider, immediately obtains every user's real password. Because people frequently reuse passwords across services, a single plaintext leak can compromise accounts well beyond the chat application itself. Hashing converts each password into a fixed-length digest using a one-way function, so that even if the credential file is exposed, the original passwords cannot be recovered directly from it.

2.3 Threats Addressed in This Assignment

Beyond basic credential theft, a multi-client network service is exposed to several other practical threats that Assignment 7 addresses directly: session hijacking through duplicate concurrent logins under the same identity, brute-force password guessing through unlimited login attempts, indefinite idle sessions that remain authenticated long after a user has walked away, and malformed or oversized input intended to disrupt the server or other clients. Each of these is mitigated by one of the six security features described in Section 6.

3. Software and System Requirements

The following software components were used to design, implement, and test the secure chat application. All components are carried over from Assignment 6, with `hashlib` added for password hashing.

Component	Purpose
Ubuntu Linux (VirtualBox)	Host operating system for running the Mininet emulated network
Python 3	Programming language used for both server and client implementation
Tkinter	Standard Python GUI toolkit used to build the client interface
Socket Programming (socket module)	Provides the TCP client and server communication primitives
hashlib (SHA-256)	Hashes passwords for secure credential storage and comparison

Component	Purpose
json / re modules	Persist hashed credentials and validate usernames with regular expressions
Mininet	Emulates a network topology with one server host and multiple client hosts
Wireshark	Captures and analyses TCP packets exchanged on port 5000
Visual Studio Code	Code editor used for writing and debugging server.py and client_gui.py

4. Network Topology

The application was tested using a Mininet-emulated network created with the command shown below, which produces a single-switch star topology connecting the server host to multiple client hosts.

```
sudo mn --topo single,5
```

Connectivity between hosts was verified using the nodes, net, and pingall commands within the Mininet CLI before starting the server and client applications. The server (server.py) was started on the host acting as 10.0.0.1, and GUI clients (client_gui.py) connected as 10.0.0.2 and 10.0.0.3 for the users bhargav and admin, consistent with the addresses recorded in security_log.txt.

```

bhargav-jyoti-saikia@bhargav-jyoti-saikia-VirtualBox: ~/SDN/CSB24022_BHARGAVJYOTISAIKIA_ASSIGNMENT7$ python3 server.py
bhargav-jyoti-saikia@bhargav-jyoti-saikia-VirtualBox: ~/SDN/CSB24022_BHARGAVJYOTISAIKIA_ASSIGNMENT7$
bhargav-jyoti-saikia@bhargav-jyoti-saikia-VirtualBox: ~/SDN/CSB24022_BHARGAVJYOTISAIKIA_ASSIGNMENT7$ python3 server.py
=====
SECURE TCP CHAT SERVER STARTED
Listening on 0.0.0.0:5000
Session timeout: 10.0 minutes
Max login attempts: 5
=====
SECURITY: [2026-07-15 12:14:21] SERVER_START | User: SYSTEM | Server started on 0.0.0.0:5000

```

Figure 4.1: Server terminal showing startup configuration — 10-minute session timeout and a 5-attempt login lockout policy — immediately before accepting client connections.

5. Authentication Flow

Assignment 7 extends the Assignment 6 wire protocol with an authentication handshake that runs before any chat message is accepted. The sequence is summarised below and implemented identically on both client_gui.py and server.py.

1. Server -> Client : "LOGIN"
2. Client -> Server : "<username>|<password>"
3. Server validates the username format, then compares the SHA-256 hash of the supplied password against users.json
4. Server -> Client : AUTH_FAILED: <reason> (invalid credentials)
or DUPLICATE_LOGIN: <reason> (user already active)
or a welcome banner + full chat access

5.1 Login Window

The login window builds directly on the Assignment 6 login_frame, adding a masked Password entry (show="*") beneath the existing Server IP and Username fields. The auth_status_label widget was introduced specifically to surface server-side authentication feedback (AUTH_FAILED, DUPLICATE_LOGIN, or a blocked-account message) separately from the general connection status label.

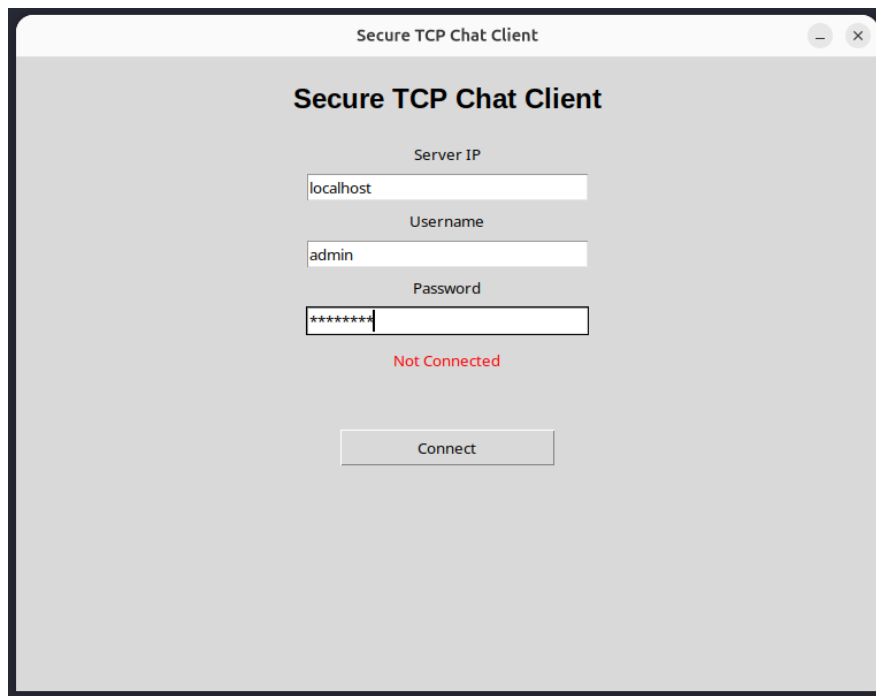


Figure 5.1: Extended login window with a masked Password field alongside Server IP and Username.

5.2 Successful Authentication

Once the server confirms the credentials, prevents a duplicate session, and admits the client, `connect_server()` switches the visible frame from `login_frame` to `chat_frame` exactly as in Assignment 6, and starts the background `receive()` thread. The chat window additionally displays a live session countdown (Session: MM:SS) driven by `update_session_timer()`, which is new in this assignment.

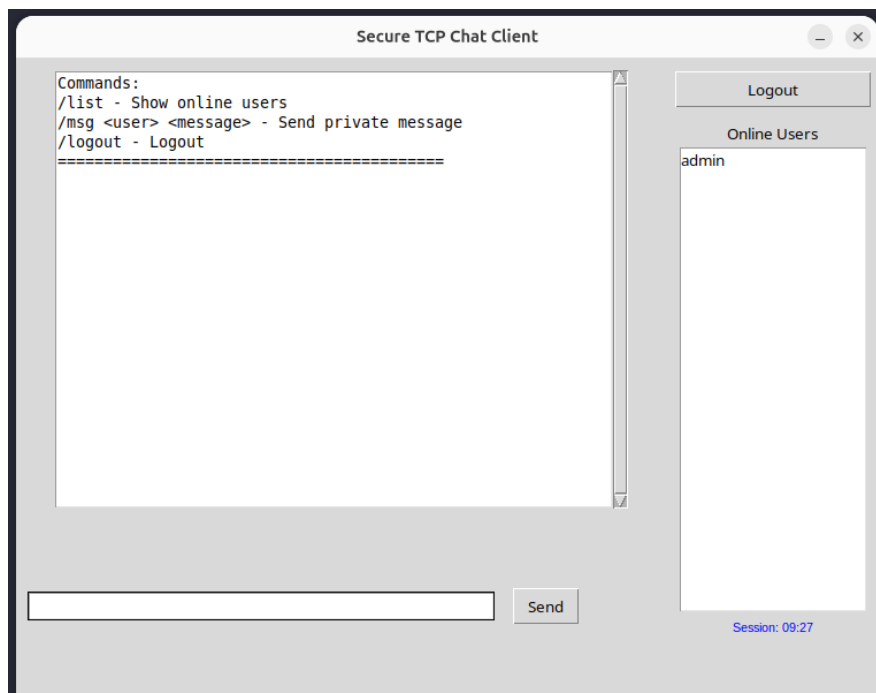


Figure 5.2: Chat window immediately after successful authentication, showing the live session countdown.

6. Security Features Implemented

All six required security tasks were layered onto the existing `server.py` and `client_gui.py` from Assignment 6. The networking logic remains entirely server-side; the GUI only reflects the server's `AUTH_FAILED`, `DUPLICATE_LOGIN`, and `LOGOUT` responses back to the user through status labels and dialog boxes.

6.1 User Authentication

On every new connection, the server sends the literal string `"LOGIN"` and waits for a single `"username|password"` line from the client. `authenticate_user()` first validates the username format, then loads `users.json` and compares the hash of the supplied password against the stored hash for that username before granting access.

```
def authenticate_user(username, password):
    valid, msg = validate_username(username)
    if not valid:
        return False, msg
    with open(USERS_FILE, "r") as f:
        users = json.load(f)
    if username not in users:
        return False, "Invalid username or password"
    ...
    if users[username] == hash_password(password):
        return True, "Login successful"
```

6.2 Secure Password Storage

Passwords are hashed with SHA-256 both when a credential is checked and, implicitly, when the initial user database is created; only the resulting hex digest is ever written to `users.json`, so the server never has to hold or compare plaintext passwords.

```
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()
```

The resulting credential store contains only hex digests, e.g.:

```
{
    "admin": "240be518fabd2724ddb6f04eeb1da...",
    "bhargav": "5e884898da28047151d0e56f8dc62...",
}
```

6.3 Duplicate Login Prevention

`prevent_duplicate_login()` checks the shared clients dictionary, protected by a `threading.Lock`, and rejects a second concurrent login for a username that already has an active, non-expired session.

```
def prevent_duplicate_login(username):
    with lock:
        if username in clients:
            if time.time() - active_sessions[username] < SESSION_TIMEOUT:
                return False, "User already logged in"
    return True, "Login allowed"
```

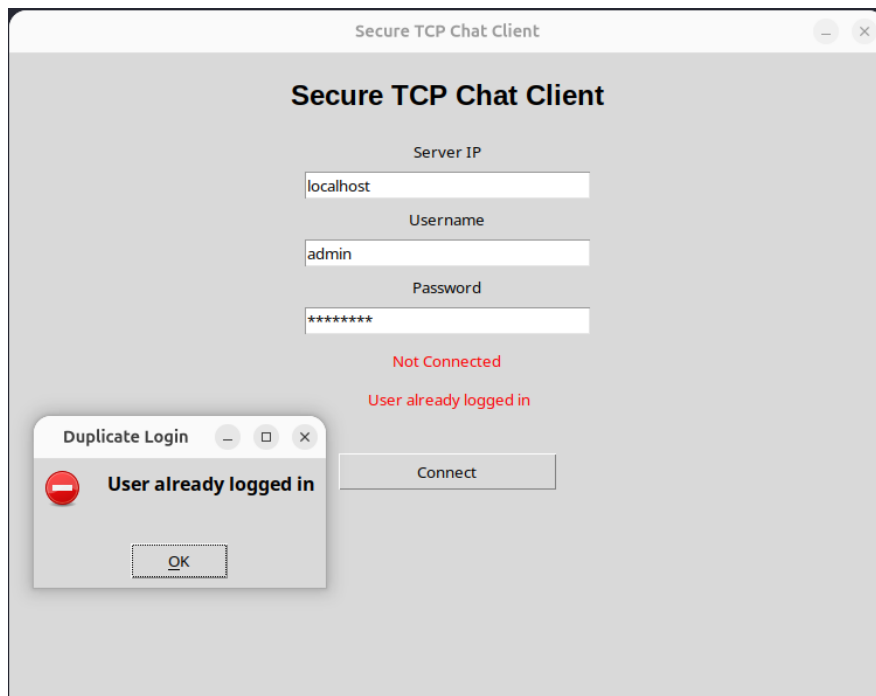


Figure 6.1: A second login attempt for an already-active username is rejected with a dedicated Duplicate Login dialog.

6.4 Input Validation

Username are restricted to 3–20 alphanumeric/underscore characters via a regular expression, chat messages are capped at 4096 characters, and a blocklist of SQL-like tokens (DROP, DELETE, --, and similar) is rejected outright. Any slash-command other than /list, /msg, and /logout is also rejected explicitly rather than being silently ignored.

```
VALID_USERNAME_PATTERN = r'^[a-zA-Z0-9_]+$'
```

```
def validate_username(username):
    if len(username) < 3 or len(username) > 20:
        return False, "Username must be 3-20 characters long"
    if not re.match(VALID_USERNAME_PATTERN, username):
        return False, "Username can only contain letters, numbers, and underscores"
    return True, "Valid username"
```

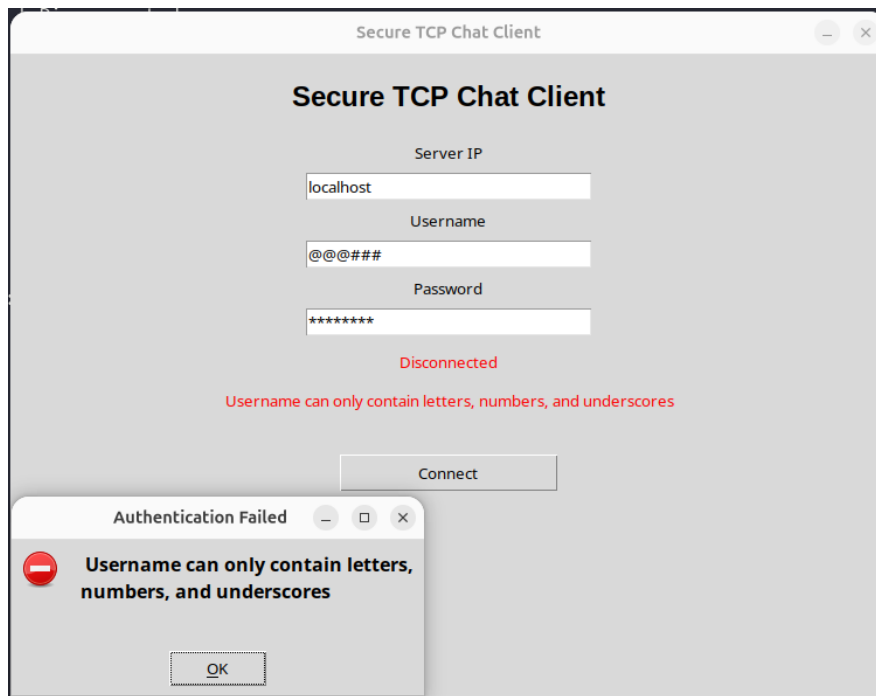


Figure 6.2: A username containing symbols (@@@###) is rejected before any credential check takes place.

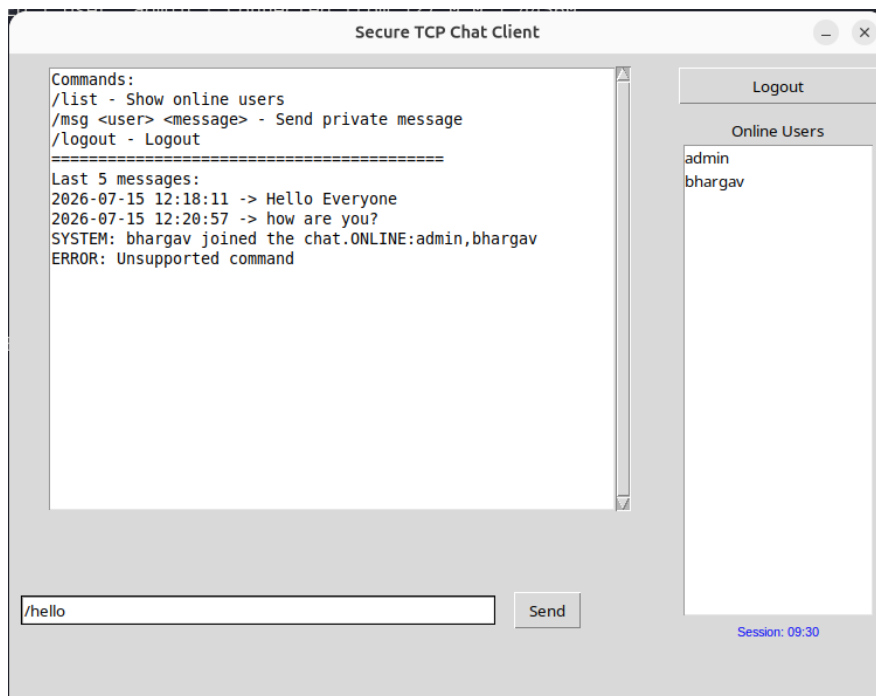


Figure 6.3: The server rejecting an unsupported /hello command with an explicit error rather than ignoring it.

6.5 Failed Login Protection

MAX_LOGIN_ATTEMPTS is set to 5 and LOGIN_BLOCK_DURATION to 300 seconds. Each failed password attempt increments a per-user counter in the login_attempts dictionary; on the fifth consecutive failure the account is locked and a blocked_until timestamp is stored and checked on every subsequent attempt until it elapses.

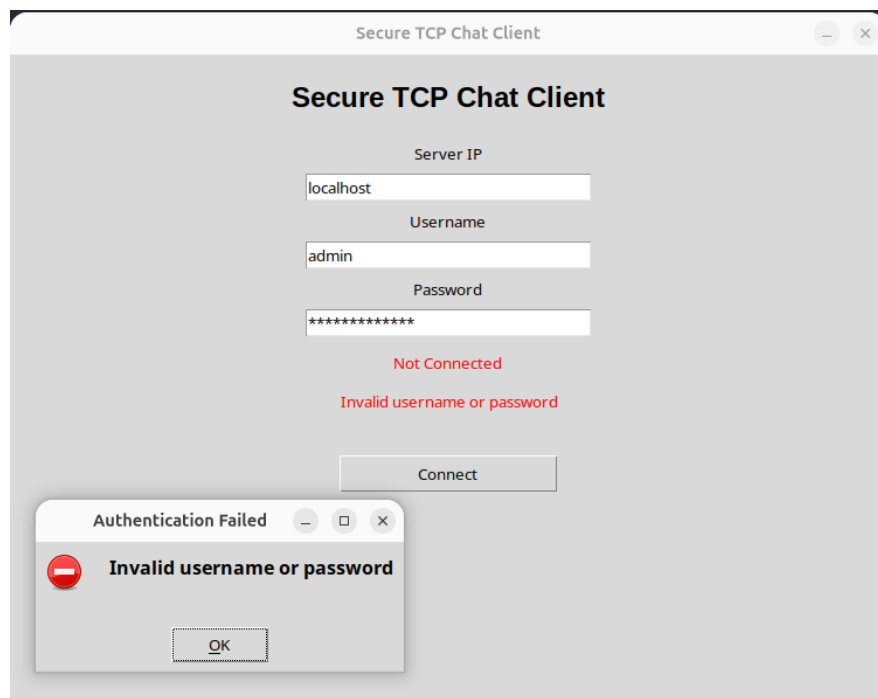


Figure 6.4: A single incorrect password is rejected with a generic “Invalid username or password” message, so the server does not reveal which field was wrong.

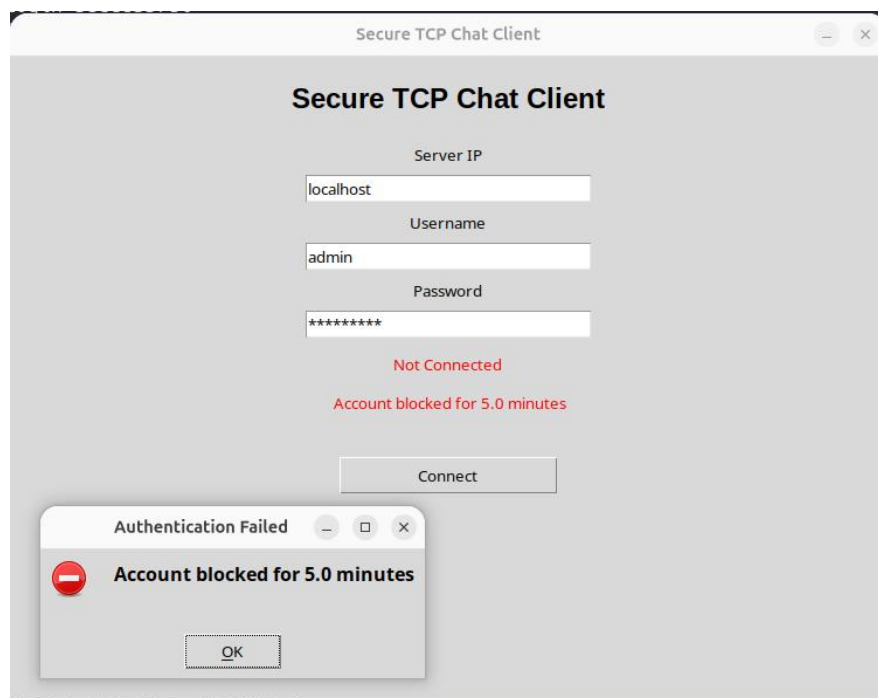


Figure 6.5: After five consecutive failed attempts, the account is temporarily blocked and the remaining cooldown is shown to the user.

6.6 Session Management and Secure Logging

SESSION_TIMEOUT is set to 600 seconds (10 minutes). The client mirrors this with a visible countdown and forces a logout when it reaches zero; the server independently tracks last-activity time in `active_sessions` and calls `logout_user()` once a session goes idle past the timeout or the user sends `/logout`. Every authentication attempt, login, logout, block, and timeout is written to `security_log.txt` by `log_security_event()`, which never includes a password field.

```
def log_security_event(event_type, username, details):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] {event_type} | User: {username} | {details}\n"
    with open(SEcurity_LOG_FILE, "a") as f:
        f.write(log_entry)
```

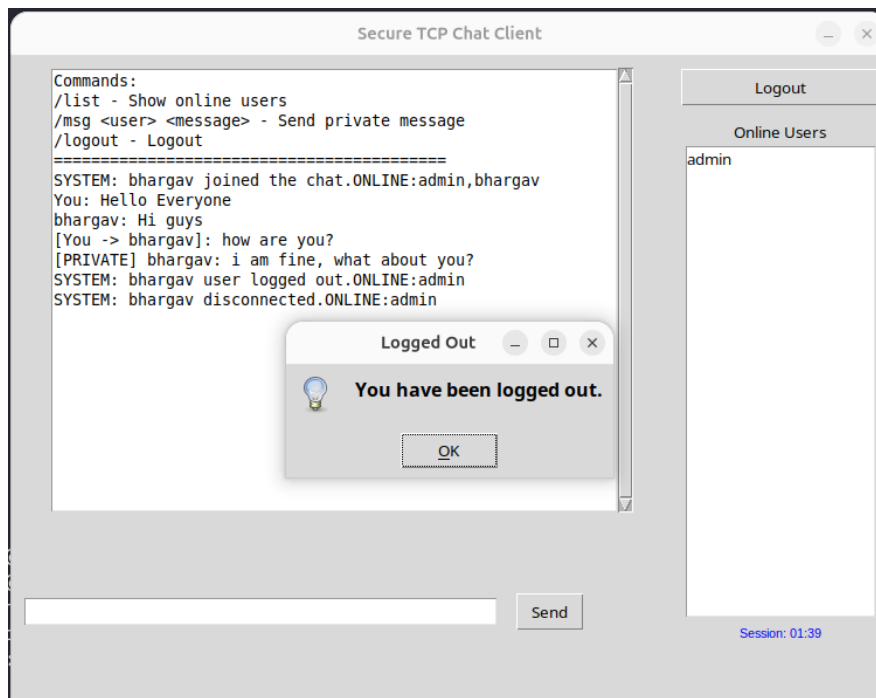


Figure 6.6: The client is notified of an automatic logout once its session exceeds the 10-minute inactivity timeout.

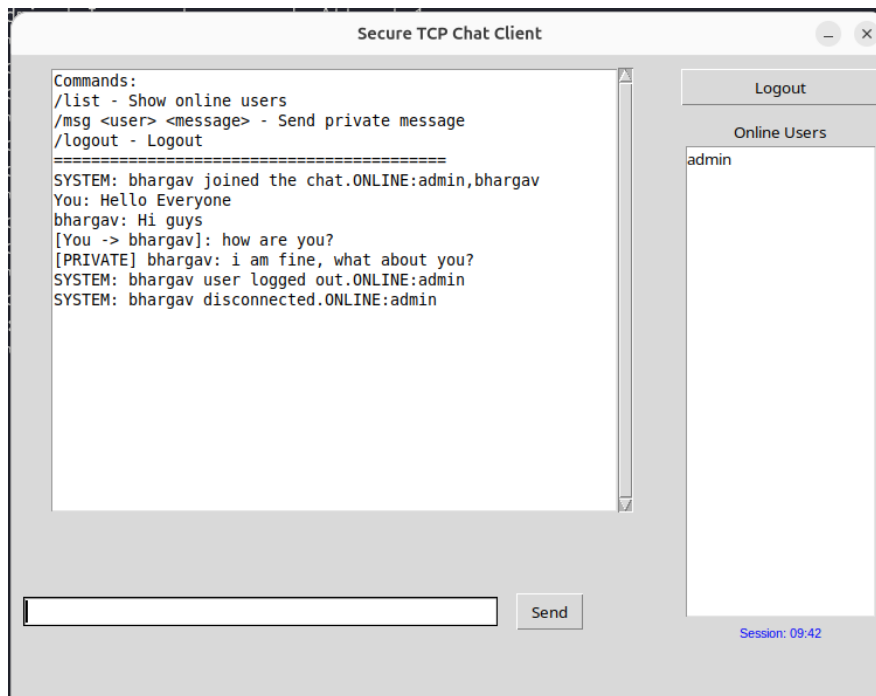


Figure 6.7: Chat log immediately before a manual /logout, showing the session running normally up to that point.

7. Components Reused from Assignment 6

In line with the assignment's requirement to extend rather than rebuild the Assignment 6 implementation, the following components were reused with no or minimal modification:

- TCP socket programming: the underlying `socket.socket(AF_INET, SOCK_STREAM)` client-server connection logic is unchanged from Assignment 6.
- Threading model: `server.py` retains the same one-thread-per-client `handle_client()` model, and `client_gui.py` retains its dedicated background `receive()` thread so the Tkinter event loop is never blocked by a socket call.
- Broadcast and private messaging: the `broadcast()` and `private_message()` functions, and the `"/msg <username> <message>"` command syntax, were carried over unchanged.
- Chat history logging: `save_history()` and the `chat_history.csv` format (timestamp, sender, receiver, message_type, message) were reused unchanged.
- Online user management: `send_online_users()` and `update_online_users()`, which push a comma-separated "ONLINE:" list to every client, were reused unchanged; the GUI's Online Users Listbox parsing logic is untouched.
- GUI structure: the `login_frame / chat_frame` layout, the `ScrolledText` chat log, the Listbox for online users, and the Send/Logout buttons all carry over from Assignment 6, extended only with a Password field and a session countdown label.

The substantial new work in this assignment is the authentication and security layer added to `server.py` (hashing, validation, lockout, session tracking, and logging) together with the corresponding feedback handling added to `client_gui.py`.

8. Implementation Design Decisions

Why SHA-256 was chosen: `hashlib.sha256()` is part of the Python standard library, requires no additional dependency, and produces a fixed-length 64-character hex digest that is simple to store as a JSON string value. Because the assignment's emphasis is on applied security engineering rather than advanced cryptography, a standard cryptographic hash was preferred over a purpose-built password-hashing scheme.

Why JSON was chosen for credential storage: `users.json` stores a simple username-to-hash mapping that Python's built-in `json` module can load and rewrite in a single call, which keeps the credential store human-inspectable (though never plaintext) and easy to extend with additional users during testing.

Why in-memory dictionaries track login attempts and sessions: `login_attempts` and `active_sessions` are plain dictionaries guarded by the same `threading.Lock` already used for the clients dictionary in Assignment 6. Because the server is expected to run for a single testing session rather than persist across restarts, in-memory tracking avoids the added complexity of a database while still providing accurate, real-time lockout and timeout behaviour.

Why a blocklist is used for message validation: `validate_message()` rejects messages containing a small set of SQL-like tokens (`DROP`, `DELETE`, `--`, and similar) in addition to enforcing the 4096-character cap. This is a lightweight, easily explained safeguard appropriate to the assignment's scope, rather than a full injection-prevention framework, since the chat server does not use a SQL backend.

Why authentication failures return a generic message: both invalid usernames and invalid passwords that pass username validation return the same "Invalid username or password" text. This prevents an attacker from using the error message to determine whether a given username exists on the server, a standard practical safeguard against username enumeration.

9. Testing Results

The application was tested inside the Mininet single,5 topology, first with clients on 127.0.0.1 during local development and later across the emulated hosts (10.0.0.2, 10.0.0.3) once the topology was brought up. The results of all functional and security test cases are summarised in Table 9.1.

Test Case	Description	Result
Valid Login	Client connects with a registered username and correct password; session begins and the countdown starts	PASS
Invalid Login	Incorrect password is rejected with a generic AUTH_FAILED message	PASS

Test Case	Description	Result
Duplicate Login	A second login for an already-active username is rejected	PASS
Invalid Username	A username containing symbols is rejected before any credential check	PASS
Failed Login Lockout	Five consecutive incorrect passwords trigger a 5-minute account block	PASS
Unsupported Command	A command other than /list, /msg, /logout is rejected with an explicit error	PASS
Session Timeout	An idle session past 10 minutes is automatically logged out	PASS
Secure Logging	Every authentication and session event is timestamped in security_log.txt with no password fields	PASS
Broadcast/Private Messaging	Existing Assignment 6 chat functionality continues to work for authenticated users	PASS

Table 9.1: Summary of functional and security test cases and results.

9.1 Security Log Walkthrough

security_log.txt records every authentication and session event generated during testing. The key cases, drawn directly from the log, are summarised in Table 9.2.

Time	Event	Outcome
12:16:13 – 12:17:41	admin and bhargav log in normally	LOGIN_SUCCESS, sessions started
12:23:50	admin attempts a second concurrent login	DUPLICATE_LOGIN_ATTEMPT rejected
12:25:52 – 12:27:00	admin enters the wrong password 5 times in a row	Attempts 1–4 → AUTH_FAILED; attempt 5 → ACCOUNT_BLOCKED for 5.0 minutes
12:30:58	admin's connection sits idle past the timeout window	SESSION_TIMEOUT → automatic logout
12:32:14	Login attempted with username "@@@@###"	Rejected: "Username can only contain letters, numbers, and underscores"
13:22:10 – 13:23:11	bhargav enters the wrong password 5 times in a row	Same 5-attempt lockout behaviour reproduced for a second account

Table 9.2: Selected security events from security_log.txt.

9.2 Functional Chat Verification

With authentication passing, chat_history.csv confirms that both broadcast and private messaging continued to work exactly as in Assignment 6. Figure 9.1 shows admin and bhargav exchanging a broadcast greeting once both are authenticated, with the Online Users list correctly populated.

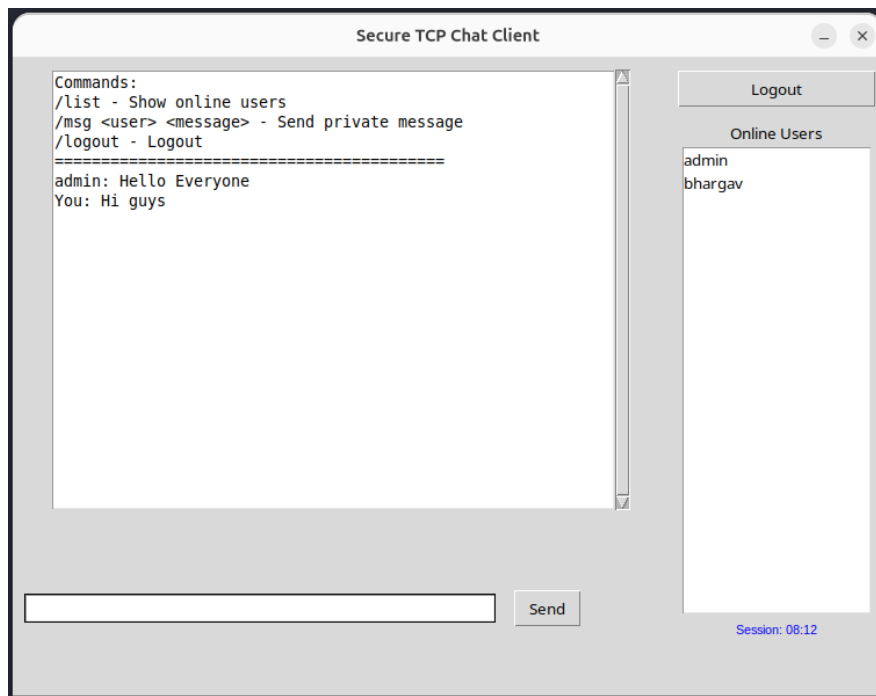


Figure 9.1: Broadcast message delivered to all authenticated clients.

Figure 9.2 shows the private message exchange (“how are you?” / “i am fine, what about you?”) between admin and bhargav, tagged [PRIVATE] on the receiving end and echoed as “[You -> user]” on the sending end, matching the chat_history.csv record for this exchange.

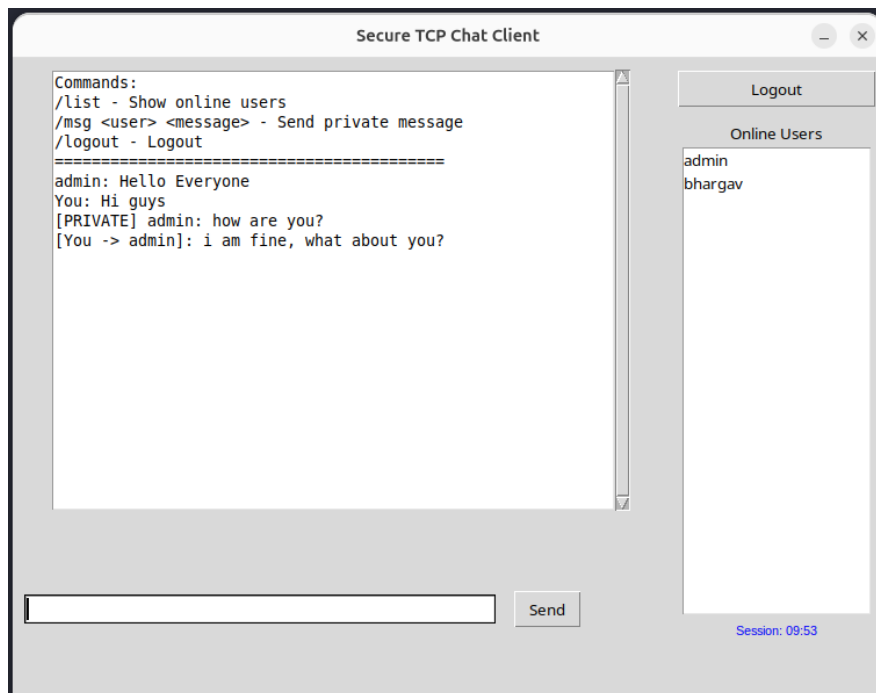


Figure 9.2: Private message exchange between admin and bhargav, visible only to the two participants.

The Online Users panel, shown cropped in Figure 9.3, continues to refresh live as clients authenticate and disconnect, exactly as in Assignment 6.



Figure 9.3: Live Online Users list showing both authenticated clients.

9.3 Handling Client-Side Feedback

client_gui.py surfaces every server-side rejection directly to the user: AUTH_FAILED and DUPLICATE_LOGIN prefixes are shown in a red auth_status_label and a message box, an account block shows the remaining cooldown in minutes, and a LOGOUT push from the server (on timeout) closes the chat window and returns to the login screen automatically, matching the behaviour captured in Figures 5.1, 6.1, 6.4, 6.5, and 6.6.

10. Wireshark Verification

Network traffic was captured on the switch interface s1-eth1 using the display filter tcp.port == 5000, which isolates all TCP traffic exchanged between the chat clients and the server across hosts 10.0.0.1 (server), 10.0.0.2, and 10.0.0.3 (clients). The following subsections present and explain the captured evidence for server initialisation, connection establishment, authenticated messaging, and connection termination.

10.1 Server Initialization

Figure 10.1 shows the ARP resolution between the Mininet hosts and the server immediately after startup, followed by the initial TCP segments exchanged once the first client connects and the LOGIN prompt and credentials are transmitted.

170	512.242388181	10.0.0.1	10.0.0.3	TCP	66	5000 → 60626 [ACK] Seq=6 Ack=16 Win=43520 Len=0 TSval=341672600 TSecr=3897803747
171	512.243589670	10.0.0.1	10.0.0.3	TCP	111	5000 → 60626 [PSH, ACK] Seq=8 Ack=16 Win=43520 Len=45 TSval=341672601 TSecr=3897803747
172	512.243798163	10.0.0.1	10.0.0.3	TCP	66	5000 → 60626 [FIN, ACK] Seq=51 Ack=16 Win=43520 Len=0 TSval=341672601 TSecr=3897803747
173	512.247312067	10.0.0.1	10.0.0.3	TCP	66	[TCP Retransmission] 5000 → 60626 [FIN, ACK] Seq=51 Ack=16 Win=43520 Len=0 TSval=341672605 TSecr=3897803747
174	512.247527343	10.0.0.3	10.0.0.1	TCP	78	60626 → 5000 [ACK] Seq=16 Ack=52 Win=42496 Len=0 TSval=3897803752 TSecr=341672601 SLE=51 SRE=52
175	516.897776208	10.0.0.3	10.0.0.1	TCP	66	5000 → 60626 [FIN, ACK] Seq=16 Ack=52 Win=42496 Len=0 TSval=3897803752 TSecr=341672601
176	516.027803297	10.0.0.1	10.0.0.3	TCP	66	5000 → 60626 [ACK] Seq=52 Ack=17 Win=43520 Len=0 TSval=341676385 TSecr=3897807532

Figure 10.6: Retransmitted FIN/ACK segments when a connection is closed after a blocked login attempt.

10.6 TCP Communication Over Port 5000

Across all captured frames, every segment exchanged between the clients and the server consistently uses source or destination port 5000, confirming that all traffic, including the authentication handshake, broadcasts, private messages, and disconnections, is carried over the single well-known port on which server.py listens.

11. Handwritten Reflection

As required by the assignment brief, the reflection questions for this assignment (covering authentication versus authorization, the rationale for password hashing, and suggested further security improvements) were answered by hand, scanned, and submitted separately as `handwritten_reflection.pdf` alongside this report, rather than being typed here.

12. Conclusion

Assignment 7 successfully layered practical application-level security onto the existing Assignment 6 chat system without altering its core socket-handling design. Passwords are hashed with SHA-256 and never stored or logged in plaintext, duplicate sessions and malformed input are actively rejected, repeated failed logins trigger a temporary account block, and idle sessions are timed out automatically. `security_log.txt` provides a complete, password-free audit trail of these events, and the existing GUI and chat functionality, broadcast and private messaging, continued to work unchanged for authenticated users.

Testing within the Mininet single,5 topology confirmed correct behaviour for valid and invalid login attempts, duplicate login prevention, input validation, failed-login logout, session timeout, and secure logging, and Wireshark packet captures on `tcp.port == 5000` verified the corresponding TCP-level behaviour for the authentication handshake, authenticated messaging, and connection teardown. All assignment objectives were successfully achieved.

13. References

- Python Software Foundation, “Python 3 Documentation,” <https://docs.python.org/3/>
- Python Software Foundation, “hashlib — Secure hashes and message digests,” <https://docs.python.org/3/library/hashlib.html>
- Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” <https://docs.python.org/3/library/tkinter.html>
- Mininet Project, “Mininet Documentation,” <http://mininet.org/>
- Wireshark Foundation, “Wireshark User's Guide,” <https://www.wireshark.org/docs/>

1. Explain the difference between authentication and authorization.

⇒ Authentication is the process of verifying the identity of a user by checking credentials such as a username and password. It answers the question, "Who are you?"

Authorization is the process of determining what an authenticated user is allowed to do. It answers the question, "What are you allowed to access?"

Authentication happens before authorization.

2. Why should passwords never be stored in plain text?

⇒ Passwords should never be stored in plain text because anyone who gains access to the database can read and misuse them. Plain-text passwords can lead to unauthorized access and identity theft. Storing passwords securely protects users even if the database is compromised.

3. What is the purpose of password hashing?

⇒ Password hashing converts a password into a fixed-length encrypted value called a hash. The original password cannot be easily recovered from the hash. During login, the entered password is hashed and compared with the stored hash. This improves security by preventing the storage of actual passwords.

4. How does duplicate login prevention improve security?

⇒ Duplicate login prevention ensures that the same user account be logged in from multiple devices or sessions at the same time. This reduces the risk of unauthorized account sharing and prevents attackers from using

stolen credentials while the legitimate user is already logged in.

5. Suggest two additional security improvements for your application.

⇒ 1) End-to-end encryption: Encrypt all chat messages so that only the sender and receiver can read them, even if the data is intercepted.

2) Two-factor authentication (2FA): Require a second verification step, such as a one-time password (OTP), in addition to the username and password to provide stronger account security.